TOPIC 6: ARRAY (ONE-DIMENSIONAL ARRAY)

At the end of the chapter, the students should be able to:

- *Define array*
- *Know how to declare and manipulate data into array*

6.1 INTRODUCTION

An array is a collection of a fixed number of components all of the same data type. A one-dimensional array is an array in which the components are arranged in a list form.

The general form for declaring a one-dimensional array is:

```
dataType arrayName[intExp];
```

in which *intExp* specifies the number of components in the array.

Array is a structured data type.

The statement:

int num[5];

declares an array *num* of five components. Each component is of type *int*. The components are *num[0]*, *num[1]*, *num[2]*, *num[3]* and *num[4]*. Figure 6.1 illustrates the array *num*.

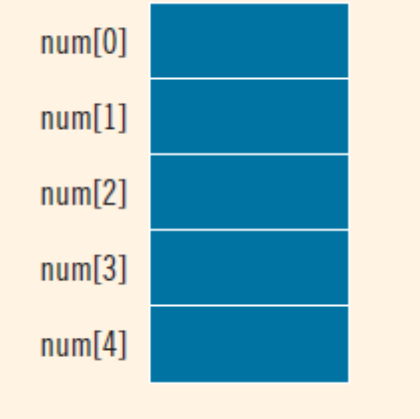

Figure 6.1 Array num

6.2 ACCESSING ARRAY COMPONENTS

The general form (syntax) used for accessing an array component is:

```
arrayName[indexExp]
```

in which *indexExp*, called the index, is any expression whose value is a nonnegative integer. The index value specifies the position of the component in the array.

In C++, *[]* is an operator called the array subscripting operator. Moreover, in C++, the array index starts at 0.

Consider the following statement:

int list[10];

This statement declares an array list of 10 components. The components are *list[0]*, *list[1]*, . . ., *list[9]*. In other words, we have declared 10 variables (see Figure 6.2).

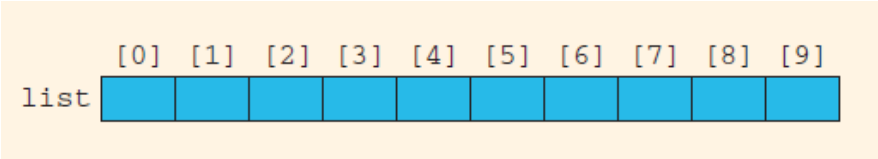

Figure 6.2 Array *list*

The assignment statement:

list[5] = 34;

stores *34* in *list[5]*, which is the sixth component of the array list (see Figure 6.3).

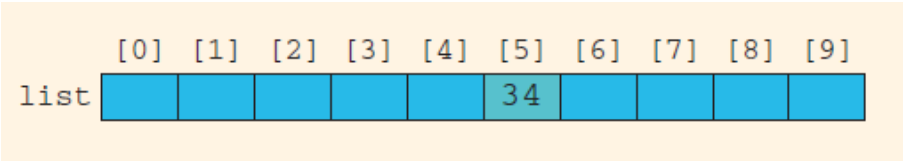

Figure 6.3 Array *list* after execution of the statement *list[5]= 34;*

6.3 PROCESSING ONE-DIMENSIONAL ARRAYS

Some of the basic operations performed on a one-dimensional array are initializing, inputting data, outputting data stored in an array, and finding the largest and/or smallest element. Moreover, if the data is numeric, some other basic operations are finding the sum and average of the elements of the array. Each of these operations requires the ability to step through the elements of the array. This is easily accomplished using a loop.

For example, suppose that we have the following statements:

```
int list[100]; //list is an array of size 100
int i;
```

The following *for* loop steps through each element of the array *list*, starting at the first element of *list*:

```
for (i = 0; i < 100; i++) //Line 1
//process list[i] //Line 2
```

If processing the *list* requires inputting data into *list*, the statement in Line 2 takes the form of an input statement, such as the *cin* statement.

For example, the following statements read 100 numbers from the keyboard and store the numbers in *list*:

```
for (i = 0; i < 100; i++) //Line 1
cin >> list[i]; //Line 2
```

Similarly, if processing *list* requires outputting the data, then the statement in Line 2 takes the form of an output statement. Figure 6.4 further illustrates how to process one-dimensional arrays.

```
//Program to read five numbers, find their sum, and
//print the numbers in reverse order.

#include <iostream>

using namespace std;

int main()
{
    int item[5];  //Declare an array item of five components
    int sum;
    int counter;

    cout << "Enter five numbers: ";

    sum = 0;

    for (counter = 0; counter < 5; counter++)
    {
        cin >> item[counter];
        sum = sum + item[counter];
    }

    cout << endl;

    cout << "The sum of the numbers is: " << sum << endl;
    cout << "The numbers in reverse order are: ";

        //Print the numbers in reverse order.
    for (counter = 4; counter >= 0; counter--)
        cout << item[counter] << " ";

    cout << endl;

    return 0;
}
```

Figure 6.4 Codes on how to process one-dimensional array

6.4 ARRAY INITIALIZATION DURING DECLARATION

Like any other simple variable, an array can be initialized while it is being declared. For example, the following C++ statement declares an array, *sales*, of five components and initializes these components.

double sales[5] = {12.25, 32.50, 16.90, 23, 45.68};

The values are placed between curly braces and separated by commas.

When initializing arrays as they are declared, it is not necessary to specify the size of the array. The size is determined by the number of initial values in the braces. However, you must include the brackets following the array name. The previous statement is, therefore, equivalent to:

double sales[] = {12.25, 32.50, 16.90, 23, 45.68};

Although it is not necessary to specify the size of the array if it is initialized during declaration, it is a good practice to do so.

6.5 ARRAYS AS PARAMETERS TO FUNCTIONS

Now that you have seen how to work with arrays, a question naturally arises: How are arrays passed as parameters to functions?

By reference only: In C++, arrays are passed by reference only.

Because arrays are passed by reference only, you do not use the symbol *&* when declaring an array as a formal parameter.

When declaring a one-dimensional array as a formal parameter, the size of the array is usually omitted. If you specify the size of a one-dimensional array when it is declared as a formal parameter, the size is ignored by the compiler.

```
void initialize(int list[], int listSize)
{
    int count;

    for (count = 0; count < listSize; count++)
        list[count] = 0;
}
```

Figure 6.5 Arrays as arguments to function

6.6 CHARACTERS ARRAY

Character arrays are of special interest, and you process them differently than you process other arrays. C++ provides many (predefined) functions that you can use with character arrays.

Character array: An array whose components are of type *char*.

C++ provides a set of functions that can be used for C-string manipulation. The header file *cstring* describes these functions. We often use three of these functions:

(i) *strcpy* - string copy, to copy a C-string into a C-string variable—that is, assignment
(ii) *strcmp* - string comparison, to compare C-strings
(iii) *strlen* - string length, to find the length of a C-string).

Table 6.1 summarizes these functions.

Function	Effect
`strcpy(s1, s2)`	Copies the string `s2` into the string variable `s1` The length of `s1` should be at least as large as `s2`
`strcmp(s1, s2)`	Returns a value `< 0` if `s1` is less than `s2` Returns `0` if `s1` and `s2` are the same Returns a value `> 0` if `s1` is greater than `s2`
`strlen(s)`	Returns the length of the string `s`, excluding the null character

Table 6.1 Functions in manipulating characters array in C++

To use these functions, the program must include the header file *cstring* via the *include* statement. That is, the following statement must be included in the program:

#include <cstring>

Suppose you have the following statements:

```
char studentName[21];
char myname[16];
char yourname[16];
```

The following statements show how string functions work:

Statement	Effect
`strcpy(myname, "John Robinson");`	`myname = "John Robinson"`
`strlen("John Robinson");`	Returns 13, the length of the string `"John Robinson"`
`int len;` `len = strlen("Sunny Day");`	Stores 9 into `len`
`strcpy(yourname, "Lisa Miller");` `strcpy(studentName, yourname);`	`yourname = "Lisa Miller"` `studentName = "Lisa Miller"`
`strcmp("Bill", "Lisa");`	Returns a value `< 0`

`strcpy(yourname, "Kathy Brown");`
`strcpy(myname, "Mark G. Clark");`
`strcmp(myname, yourname);`
	`yourname = "Kathy Brown"`
	`myname = "Mark G. Clark"`
	Returns a value `> 0`

Figure 6.6 Explanation on how each function on character array manipulation works

6.7 READING AND WRITING STRINGS

6.7.1 String Input

To read and store a line of input, including whitespace characters, you can also use the stream function *getline*. Suppose that you have the following declaration:

char textLine[100];

The following statement will read and store the next 99 characters, or until the newline character, into *textLine*. The null character will be automatically appended as the last character of *textLine*.

cin.getline(textLine, 100);

6.7.2 String Output

The output of C-strings is another place where aggregate operations on arrays are allowed. You can output C-strings by using an output stream variable, such as *cout*, together with the insertion operator, <<. For example, the statement:

cout << name;

outputs the contents of name on the screen. The insertion operator, <<, continues to write the contents of name until it finds the null character. Thus, if the length of name is 4, the above statement outputs only four characters. If name does not contain the null character, then you will see strange output because the insertion operator continues to output data from memory adjacent to name until *'\0'* is found.